NOVA IMS
INFORMATION MANAGEMENT SCHOOL

MLOps

Master's in Data Science and Advanced Analytics

Group 10

Alexander Batista, 20250419

Mehmet Karaca, 20250344

Luis Mendes, 20221949

Veronica Mendes, 20221945

Miguel Matos, 20221925

Spring Semester 2025-2026

Table of Contents

1. Motivation and Success Metrics

Access to clean drinking water is one of the most fundamental public health challenges worldwide. According to the WHO, approximately 2 billion people lack access to safe water at home, making automated water quality assessment a practically meaningful problem. We chose the Kaggle Water Potability dataset because it represents a realistic scenario where automated screening could supplement traditional laboratory testing, particularly in resource-constrained settings where testing every sample manually is not feasible.

The dataset contains 3,276 water samples, each described by nine physicochemical measurements: pH, hardness, total dissolved solids, chloramines, sulfate, conductivity, organic carbon, trihalomethanes, and turbidity. The binary target indicates whether a sample is safe to drink (1) or not (0). We defined success along two dimensions before any modelling began:

Success threshold 1 (ROC-AUC ≥ 0.65 on the held-out test split): Because tabular water quality datasets often feature inherently noisy and heavily overlapping chemical distributions, perfect classification without overfitting is rare. ROC-AUC is threshold-independent and evaluates the model's global ranking ability without being biased by the sheer volume of the majority class. Therefore, clearing 0.65 is our headline measure proving that the model successfully extracted a statistically significant, generalizable signal well above random guessing (0.50).

Success threshold 2 (F1 ≥ 0.45): In an imbalanced safety context, the F1 score provides a holistic measure of the model's ability to balance precision and recall across both the safe and unsafe water classes. While catching unsafe water is the primary objective, practical utility requires an integrated system that also reliably identifies safe water to avoid overwhelming operational resources with false alarms. Achieving a global F1 of 0.45 confirms that the model maintains a functional, system-wide equilibrium, demonstrating that it has learned generalizable patterns across the entire dataset rather than simply defaulting to the majority class to artificially inflate standard accuracy.

We considered our pipeline successful if the best model cleared both thresholds on the untouched test split. These thresholds are reporting bars, evaluated once on the holdout after the model is chosen (see Section 3.3).

2. Project Planning

Instead of rigid time-boxed sprints, we used a data-driven, **agile Kanban methodology** governed by data dependencies mapping directly onto a Kedro Directed Acyclic Graph (DAG). Each milestone required satisfying a strict **Definition of Done (DoD)**, Git commitment, reproducible data artifacts, and passing unit tests, before downstream phases could ingest its outputs. Mirroring agile feedback loops, phases were evolutionary rather than frozen; for example, feature engineering adjustments triggered automated re-validation of upstream data contracts. The project was executed across four phases:

Phase 1: Exploratory Analysis & Data Contract: Started by exploring the dataset in `notebooks/EDA.ipynb` to map feature distributions (near-Gaussian), missingness limits,

and class balances. These insights directly shaped our parametric preprocessing. We codified these bounds into a fail-fast Great Expectations data contract node to validate inputs immediately.

Phase 2: Preprocessing Pipeline: Built an 85/15 stratified split, 23 domain features, and training-only outlier removal. To prevent data leakage, all stateful transformers (imputation, scaling, RFECV selection) were fitted strictly inside local cross-validation (CV) folds rather than globally.

Phase 3 - Modelling, Tuning and MLflow Tracking: Evaluated five model families (Logistic Regression baseline, Random Forest, Extra Trees, Histogram Gradient Boosting, XGBoost). Hyperparameters were optimized via fold-localized Optuna searches on the training set. Every run was instrumented with MLflow to track fold metrics, parameters, and serialized model files.

Phase 4 - Explainability, Drift Detection, and Serving: Generated SHAP explainability matrices for the top performer (Extra Trees). For post-deployment monitoring, we built an independent data drift pipeline using Kolmogorov-Smirnov (KS) tests and containerized the model using a FastAPI REST API and Docker.

To ensure robust data state management and strict reproducibility, the project architecture is strategically organized into three core functional pipelines: ‘preprocessing’, ‘modeling’, and ‘data_drift’; alongside an isolated, opt-in ‘tuning’ pipeline, all of which remain highly modular and can be executed independently (e.g., kedro run --pipeline data_drift). The preprocessing pipeline encompasses data validation, data cleaning, feature engineering, and data splitting; grouping these steps ensures leakage-safe state management because they share a single fitted state per cross-validation fold (incorporating the imputer, scaler, and RFECV selector), thereby eliminating the computational overhead of re-loading and re-serializing that state between arbitrary stages. Following this, the modeling pipeline handles model training, evaluation, and MLflow logging for all five model families. Crucially, this default modeling pipeline never performs hyperparameter optimization, it simply refits models using strictly committed parameters to guarantee perfectly reproducible artifacts, because hyperparameter search is intentionally isolated into its own opt-in tuning pipeline. This separation is necessary because Optuna's Tree-structured Parzen Estimator (TPE) sampler is sequential and floating-point sensitive, meaning re-tuning is not bit-reproducible across machines, so the tuning pipeline is only triggered when developers explicitly want to rewrite those parameters. Finally, the data_drift pipeline serves as a dedicated post-deployment monitoring layer, utilizing Kolmogorov-Smirnov tests to evaluate incoming datasets against the established baseline and detect any statistical shifts over time.

3. Data Exploration and Modelling Results

3.1 Exploratory Analysis

The EDA revealed three important characteristics of the dataset, visible in the feature distributions. (Annex, figure 2) An initial examination of the dataset revealed three findings with direct implications for the preprocessing and evaluation design. First, all nine physicochemical

measurements exhibit low skewness, with absolute values not exceeding 0.7, indicating near-Gaussian distributions. This property justifies the use of parametric preprocessing techniques, namely Z-score-based outlier removal, mean imputation, and standard scaling, each of which assumes approximate distributional symmetry. Second, missing values are confined to three features: pH (15.0%), sulfate (23.8%), and trihalomethanes (4.9%). This selective missingness is consistent with domain conventions, as measurement instruments occasionally produce out-of-range readings that are recorded as absent rather than zero. Missing values are addressed through mean imputation fitted exclusively on training folds, ensuring that no information from evaluation splits is introduced into the transformation pipeline. Third, the dataset exhibits moderate class imbalance, with 39% of samples labelled as potable and the remaining 61% as non-potable. Stratified splitting and cross-validation folds are employed to preserve this ratio consistently across all subsets. Furthermore, ROC-AUC is adopted as the primary evaluation metric precisely because it is threshold-invariant and robust to class imbalance of this magnitude.

3.2 Feature Engineering

Feature engineering was implemented as a deterministic, domain-informed transformation step after the stratified train-test split and before any learned preprocessing. Because all engineered variables are algebraic functions of the observed water-quality measurements, they can be generated independently for each data split, while leakage-sensitive operations are deferred to the cross-validation stage.

Starting from the nine original physicochemical measurements, 23 additional candidate predictors were constructed, resulting in a 32-feature modelling matrix. The engineered variables were organised into three categories. Ratio-based features capture relative concentration and load patterns between dissolved solids, conductivity, sulfate, hardness, organic carbon, and trihalomethanes, expressing proportional relationships that may be more informative than absolute concentrations alone. Interaction features model joint effects between variables whose combined behaviour may influence potability. Threshold-based risk indicators encode whether individual measurements exceed established water-quality guideline ranges, while aggregate risk scores summarise simultaneous threshold violations as a proxy for cumulative water-quality risk.

RFECV was subsequently applied within the learned preprocessing stage, ensuring that feature selection was performed exclusively on the active training fold. The resulting pipeline combines domain knowledge with empirical selection, retaining only the subset of chemistry-informed candidate features that demonstrably improves cross-validated predictive performance.

3.3 Model Comparison

All models were evaluated with 5-fold stratified cross-validation on the training set and a single held-out test evaluation. The table below reports the primary development metric (CV F1) alongside the test ROC-AUC, which is the success criterion defined in Section 1.

Model	CV F1 ± std	Test ROC-AUC	Test F1
Extra Trees	0.498 ± 0.044	0.666	0.574
Hist. Gradient Boosting	0.489 ± 0.043	0.682	0.574
Random Forest	0.485 ± 0.047	0.670	0.530
XGBoost	0.415 ± 0.049	0.674	0.506
Logistic Regression	0.132 ± 0.077	0.551	0.136

Table 1: *Model Comparison for Metrics*

Models are selected by the primary development metric (CV F1), computed on the training split only, before the test set is touched. This is the development selection criterion and is deliberately distinct from the ROC-AUC success threshold in Section 1: ROC-AUC is a reporting bar checked once on the holdout, never a selection signal. Extra Trees achieves the highest CV F1 (0.498) and is therefore the champion that we serve, explain, and monitor for drift; its test ROC-AUC of 0.666 and test F1 of 0.574 clear both success thresholds from Section 1. We want to be explicit that this selection is statistically fragile. The top three tree ensembles (Extra Trees 0.498, Hist. Gradient Boosting 0.489, Random Forest 0.485) all fall within one CV standard deviation (about 0.044) of the best score, so under a one-standard-error rule they are effectively tied and the ranking among them is within noise. Extra Trees is promoted because it leads on the point estimate of the primary development metric and also holds the best holdout test F1 (0.5743, fractionally above Hist. Gradient Boosting's 0.5735; both round to 0.574 in the table above), which serves as the tiebreak, while Hist. Gradient Boosting edges ahead only on test ROC-AUC (0.682). Because that margin is so small, the champion is fixed by a deliberate manual promotion gate (see Section 4.2), not derived automatically, and a re-tune that re-ranked these three would be a legitimate reason to re-promote. The logistic regression baseline, while stable (low CV std), is far behind the tree models, confirming the relationship between water quality and potability is not well captured by a linear boundary.

It is worth being direct about what "clearing the thresholds" means here: a ROC-AUC of 0.666 and an F1 of 0.574 are real but modest, closer to "somewhat better than a coin flip" than to a deployable diagnostic tool, and the same pattern holds for every model family we tried, not just ours. This is consistent with how the dataset is known to behave: the Kaggle Water Potability dataset's labels are synthetically generated and only weakly tied to the nine physicochemical features, so even an ideal classifier cannot push performance much further without additional, more informative measurements. We set the thresholds in Section 1 specifically at a level that separates "the model learned a real, non-trivial signal" from "the model is guessing," and Extra Trees clears that bar, but readers should not mistake it for a clinically usable potability test. The honest conclusion is that the production-readiness work in Section 4 (serving, drift monitoring, retraining triggers) is the more transferable outcome of this project than the absolute performance of the classifier itself.

The confusion matrix on the test split shows the model identifies non-potable samples (the majority class) somewhat more reliably than potable ones, which is expected given the class imbalance (See Annex, Figure 3).

3.4 SHAP Feature Importance

SHAP (SHapley Additive exPlanations) attributes each feature's contribution to individual predictions on a theoretically grounded basis. We used TreeExplainer, which computes exact SHAP values for tree models without approximation.

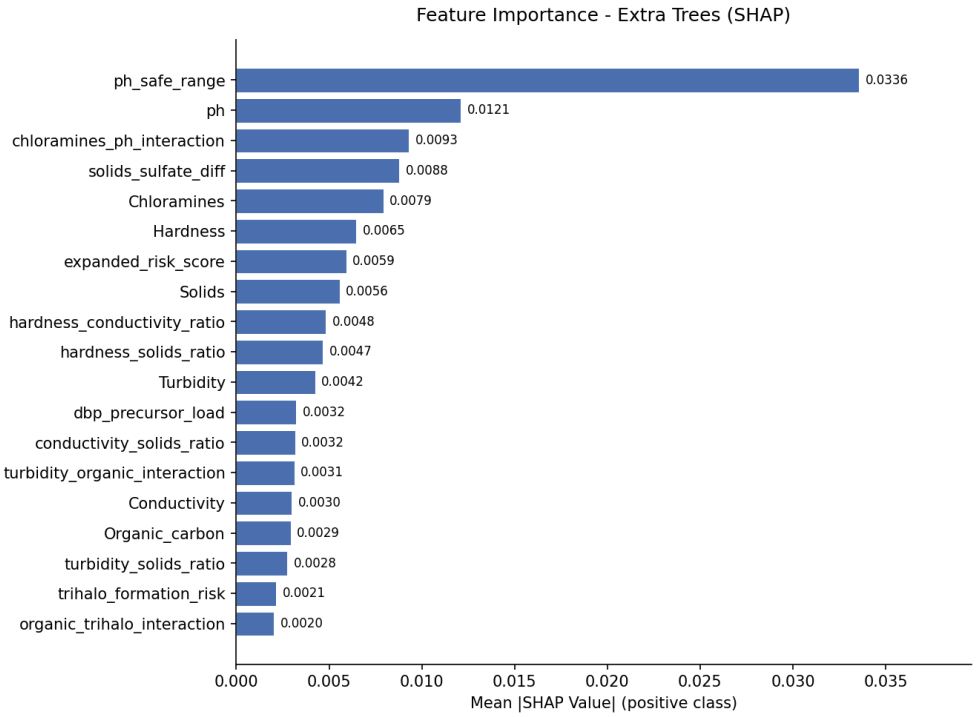

Figure 1: *Feature Importance - Extra Trees Model on test data*

pH-related features dominate, consistent with domain knowledge: pH is the single most commonly monitored indicator of water safety. The top feature is the engineered `ph_safe_range` flag (whether pH falls in the safe 6.5-8.5 band), followed by the raw `ph` value itself and the `chloramines_ph_interaction` term, which captures the combined effect of chloramine concentration and acidity since both are disinfection-related and interact chemically. That the single most important feature is engineered, together with several others in the top ranks (`chloramines_ph_interaction`, `solids_sulfate_diff`), confirms that the feature engineering step in Section 3.2 added genuine predictive value beyond the nine raw measurements.

4. Production Implementation Discussion

This proof of concept is structured so each stage, from data quality and feature engineering to training, serving, and drift detection, can run as a full end-to-end sequence or as isolated, independently triggerable Kedro pipelines, which is exactly what a real production MLOps deployment requires.

4.1 Technology Choices and Their Advantages

Kedro enables modular, selective pipeline execution in production, for example running only drift checks on a nightly schedule without triggering retraining. Great Expectations provides fail-fast data contracts at two checkpoints (raw input and model-ready features), ensuring corrupt or out-of-schema data is rejected before it silently degrades predictions. MLflow gives full auditability of every run, covering parameters, metrics, artifacts, and model versions, and would be backed in production by a centralized tracking server with a shared database and object storage. Optuna hyperparameter search is deliberately decoupled from the reproducible modeling pipeline, which is the correct production pattern: re-tuning is an explicit opt-in, not a side-effect of every run. SHAP explainability is persisted as both a CSV and a plot, making it embeddable in automated reporting, which is essential in safety-critical domains like water quality. FastAPI and Docker provide a typed, containerized prediction service with a /health endpoint compatible with standard load balancer health checks, making it portable across local, CI, and cloud environments. Finally, the KS-test drift pipeline runs as a separate, schedulable unit so it can operate against live production samples without interfering with training or serving.

4.2 Risks and Mitigations

The preprocessing and feature engineering layer is built on Pandas, which works well for the current dataset size but may not remain efficient if the volume of incoming water quality readings grows significantly. Switching the computation to a distributed framework would address this without requiring changes to the Kedro node interfaces. The FastAPI service currently runs as a single container with no autoscaling or load balancing, and in production it should be deployed behind Kubernetes HPA or a managed platform such as Cloud Run or SageMaker. MLflow tracking uses a local file-store that is not shareable across machines or teams, and the mitigation is a centralized MLflow Tracking Server backed by PostgreSQL and S3, requiring only a change to the MLFLOW_TRACKING_URI environment variable. Optuna's TPE search is not bit-reproducible across environments, so tuning should only run in a controlled CI job. The default modeling pipeline already mitigates this by always refitting from committed best parameters. To validate this monitoring, we simulated a production shift (lower pH, higher chloramines, sulfate, and trihalomethanes): the Kolmogorov-Smirnov test flagged 17 of 32 features as drifted, versus only 2 of 32 between the train and test splits, and the Extra Trees champion's test ROC-AUC fell from 0.666 to 0.564.The drift pipeline currently only reports distribution shift without triggering any action, and in production its output should be wired to a retraining process that fires automatically when the fraction of drifted features exceeds a configurable threshold. The file-based feature store lacks concurrent write support and point-in-time retrieval, but the existing load_offline_features abstraction provides a clean boundary for migrating to a managed solution such as Feast. Finally, model promotions are currently manual with no CI/CD gate, which risks deploying a degraded model. The mitigation is to add automated metric threshold checks in a pipeline before any artifact is promoted to the serving container.

Annex

Package List: The project uses Python 3.13 and manages dependencies with uv. The table below lists the key packages and their pinned versions as declared in `pyproject.toml` and resolved in `uv.lock`.

Package	Version	Purpose
Python	3.13.13	Runtime
kedro	1.3.1	Pipeline orchestration
pandas	2.3.3	Tabular data manipulation
scikit-learn	1.8.0	Model training, CV, preprocessing
scipy	1.17.1	KS-test for drift detection
numpy	2.4.4	Numerical operations
xgboost	3.2.0	Gradient boosted trees
optuna	4.8.0	Hyperparameter optimisation
mlflow	3.12.0	Experiment tracking
great-expectations	1.17.2	Raw data contract validation
shap	0.52.0	Feature importance (SHAP values)
fastapi	0.136.1	REST API for model serving
uvicorn	0.47.x	ASGI server for FastAPI
matplotlib	3.10.9	Confusion matrix and SHAP plots
kaggle	2.1.2	Dataset download
pytest	9.0.3	Unit and integration testing

Table 2: *Package List*

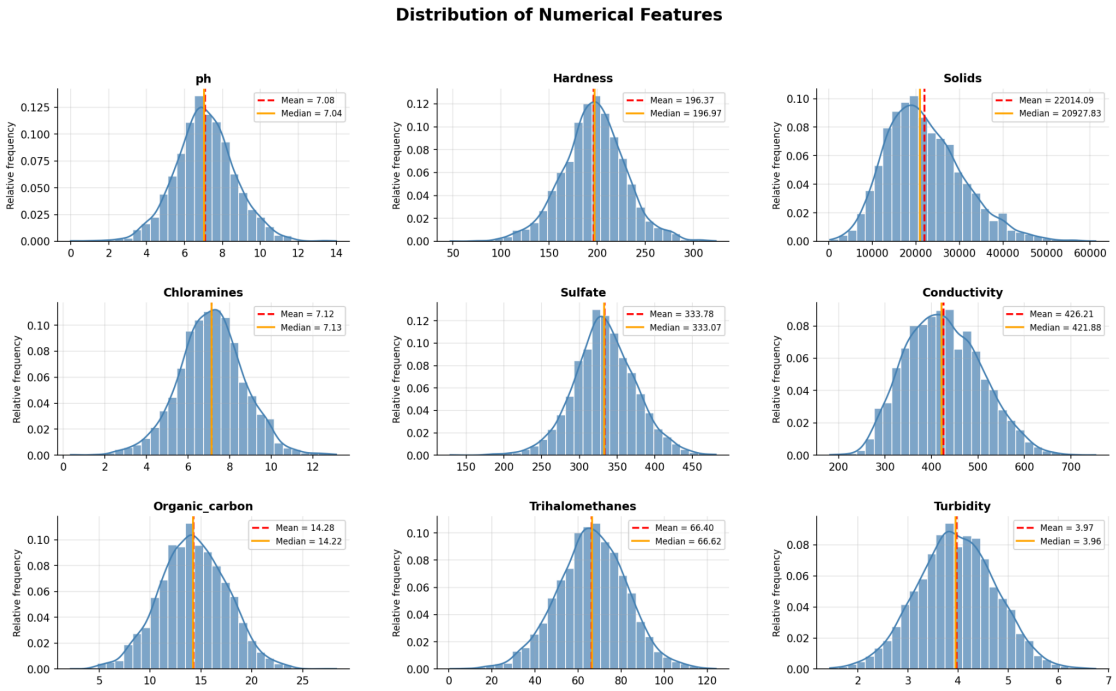

Figure 2: *Distribution of Numerical Features of the dataset*

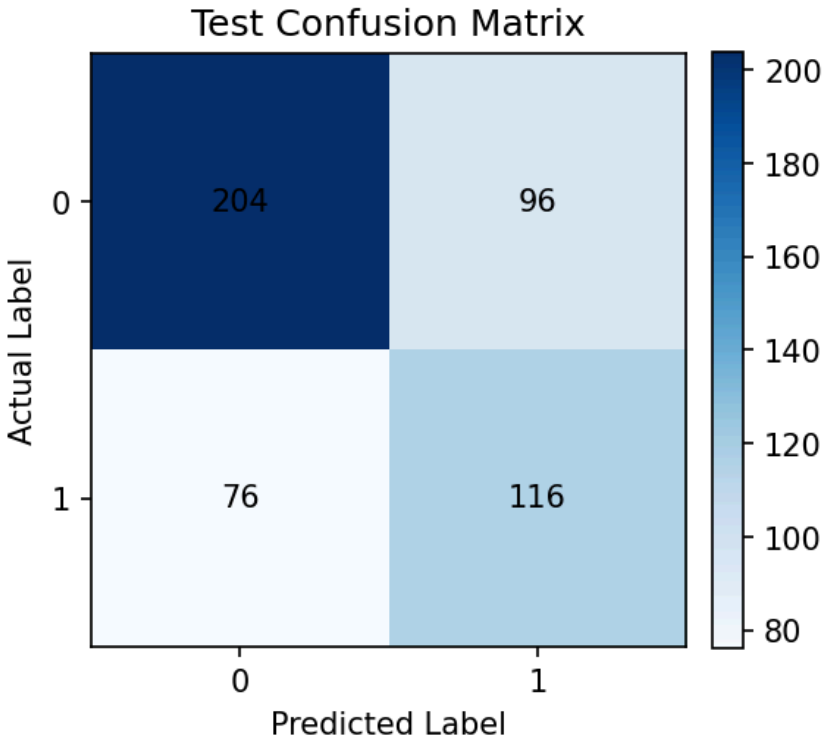

Figure 3: *Confusion Matrix - Extra Trees on test data*